

Real Time Waste Detection

Marzo Zennaro
Mid Sweden University
Sundsvall, Sweden
maze2501@student.miun.se

Javier Jerez Reinoso
Mid Sweden University
Sundsvall, Sweden
jaje2501@student.miun.se

Abstract—This project investigates real-time waste detection using YOLO-based object detectors trained on the TACO (Trash Annotations in Context) dataset. Several dataset configurations, including category, supercategory and binary representations, were evaluated to analyze the impact of class imbalance on detection performance. Multiple architectures from the YOLOv8 and YOLO26 families were compared under different training strategies, image resolutions and transfer learning configurations. The results show that the binary dataset significantly outperformed the multi-class alternatives, while higher input resolutions and backbone freezing improved overall detection accuracy. The best-performing model, YOLO26m with a frozen backbone and a resolution of 960 pixels, achieved a mAP50 of 0.561 while maintaining real-time inference capabilities. The study highlights the challenges posed by small objects, limited dataset size and severe class imbalance.

Index Terms—Object Detection, YOLO, YOLOv8, YOLO26, TACO Dataset, Waste Detection, Deep Learning, Computer Vision.

I. INTRODUCTION

Litter accumulation in urban and natural environments is an important environmental challenge. Automatic waste detection systems can support applications such as environmental monitoring, smart recycling and autonomous cleaning.

Unlike image classification, object detection identifies both the category and location of objects through bounding boxes. This capability is essential for waste detection, where multiple objects may appear simultaneously within the same image.

This project investigates the use of YOLO-based object detectors for real-time waste detection using the TACO (Trash Annotations in Context) dataset [1]. The study evaluates different YOLO architectures under challenging real-world conditions, including cluttered backgrounds, object occlusions, scale variations and class imbalance.

The main objective is to assess the effectiveness of transfer learning for waste detection and to analyze how dataset characteristics influence model performance. Detection accuracy, computational cost and real-time deployment feasibility are considered throughout the evaluation.

II. DATASET

The experiments were conducted using the TACO (Trash Annotations in Context) dataset, a publicly available benchmark for waste object detection [1]. The dataset contains approximately 1,500 real-world images annotated with bounding boxes corresponding to various categories of litter and waste objects.

The images were collected in unconstrained outdoor environments and contain challenging conditions such as cluttered backgrounds, partial occlusions, varying illumination and substantial differences in object scale. These characteristics make the dataset representative of realistic waste detection scenarios but also increase the difficulty of the detection task.

The dataset includes a large variety of waste categories and urban litter, as well as another distribution of the same objects in a wider set of categories, named 'supercategories', which group the data in fewer labels.

A strong class imbalance can be observed across the dataset, where a small number of categories account for a large proportion of the annotations, while several classes contain only a limited number of examples. Categories such as cigarette waste (around 650 samples), unlabeled litter (around 500 samples) and plastic film (850 samples) are highly represented, whereas classes such as batteries, blister packs and jars contain very few annotated instances. This imbalance can introduce bias during training and negatively affect performance on minority classes.

Another important observation is the frequency of small objects. Many waste items occupy only a small portion of the image area, see Fig. 1, making localization particularly challenging. Since object detection performance is commonly evaluated using Intersection over Union (IoU), even small localization errors can significantly reduce evaluation scores for these objects.



Fig. 1. Dataset example of waste segmentation.

The dataset also exhibits considerable variation in image resolution and aspect ratio. Consequently, an appropriate pre-processing pipeline is required to ensure consistent model inputs while preserving as much visual information as possible.

III. METHODOLOGY

The project focused on the YOLO family of object detectors due to their strong balance between detection accuracy and real-time inference speed. Since one of the objectives of the project was to evaluate the feasibility of real-time waste detection, computational efficiency was considered a critical design requirement in addition to detection performance.

A. Model Selection

The YOLO architecture can be conceptually divided into three main components:

- **Backbone:** Responsible for feature extraction.
- **Neck:** Aggregates multi-scale feature representations.
- **Detection head:** Predicts bounding boxes and object classes.

Among the different YOLO generations, YOLOv8 [4] was selected as the primary baseline architecture. Compared to earlier versions such as YOLOv5, YOLOv8 [4] provides several improvements in training stability, feature extraction, optimization efficiency.

More recent architectures such as YOLOv10 or YOLO11 were not selected for the initial experiments. Although these models may provide additional optimizations, YOLOv8 currently offers stronger documentation, reproducibility and wider community support, making it more suitable for evaluation project.

B. YOLOv8 and YOLOv26 Variants

The YOLO architectures evaluated during the project include both small and medium-scale variants:

- YOLOv8s
- YOLOv8m
- YOLO26s
- YOLO26m

Smaller architectures such as YOLOv8s are optimized for faster inference and lower memory consumption, making them attractive for real-time deployment scenarios. Medium-sized architectures such as YOLOv8m introduce additional parameters and feature extraction capacity, potentially improving detection performance on visually complex datasets.

After observing limited improvements between YOLOv8s and YOLOv8m, additional experiments considered the YOLO26 family [5]. Despite the fact that YOLO26 currently has less documentation and fewer available references, it introduces architectural modifications specifically designed to improve feature extraction and object representation efficiency, as well as small objects detection.

However, increasing model complexity also increases training time, memory consumption and the risk of overfitting, particularly when training data is limited. Since the TACO dataset contains only approximately 1,500 images, evaluating both small and medium variants allows the project to analyze whether larger models provide meaningful performance improvements or whether dataset limitations become the dominant bottleneck.

C. Metrics

Model efficiency and computational requirements were analyzed using several criteria, including:

- Detection accuracy metrics (mAP50, mAP50-95)
- Precision and recall
- Inference time per image

Inference speed was considered particularly important because the final application scenario targets real-time waste detection systems. Consequently, the project evaluates not only detection accuracy but also whether the models can realistically satisfy real-time constraints.

D. Transfer Learning and Layer Freezing

Training deep neural networks entirely from scratch requires very large datasets and substantial computational resources. Since the TACO dataset contains only approximately 1,500 images [1], transfer learning was adopted as the primary training strategy.

The models were initialized using weights pretrained on the COCO dataset [3] and subsequently fine-tuned on the waste detection task. This allows the network to reuse low-level visual features such as edges, textures and shapes learned from large-scale datasets.

The project also investigated layer freezing strategies, where selected pretrained layers remain fixed during training while the remaining layers are updated. Freezing the backbone can preserve generic visual representations learned from COCO while reducing training instability and overfitting on small datasets. In contrast, full fine-tuning updates all model parameters and allows the network to adapt more strongly to the target domain, although this increases training time and the risk of overfitting.

IV. TRAINING PIPELINE

The entire training and evaluation pipeline was implemented using the Ultralytics YOLO framework built on top of PyTorch [2]. The framework provides an integrated environment that simplifies model experimentation and training management.

Ultralytics automatically handles several critical components of the training pipeline, including:

- Dataset loading and batching.
- Data augmentation.
- Image resizing and normalization.
- Optimizer and scheduler management.
- Validation and checkpointing.
- Metric logging and visualization.

The framework also provides built-in support for transfer learning using pretrained YOLO weights, enabling rapid experimentation with different architectures and hyperparameter configurations.

An additional advantage of the framework is its support for layer freezing strategies. During training, selected pretrained layers can remain frozen while the remaining layers are updated.

The complete hyperparameter configurations across all training runs are summarized in Table III in the Appendix.

A. Data preparation

The original TACO annotations were converted into YOLO-compatible format by normalizing the bounding box coordinates according to the YOLO annotation standard.

The dataset was divided into training, validation and test subsets. The training set was used for parameter optimization, the validation set was used for monitoring convergence and early stopping and the test set was reserved for the final performance evaluation. Since Ultralytics YOLO [2] requires a specific dataset structure and configuration file for training, we had to build a specific `dataset.yaml` file for each processed dataset, which defines the dataset paths and class configuration used by the model. Furthermore, the three different dataset configurations that were explored during the project are the following:

- Binary dataset (trash / no trash).
- Full-category dataset (all original TACO categories).
- Supercategory dataset (grouped categories such as plastic, paper, cans and bottles).

The initial experiments were performed using the original categories and supercategories. However, the dataset presented a severe class imbalance problem, where several classes contained too few training examples for stable deep learning training. This made robust multi-class object detection difficult under the available dataset size constraints. Hence, to address this limitation, later experiments focused primarily on the binary detection task, simplifying the classification problem while preserving the core objective of real-time waste localization and detection.

B. Training Infrastructure

The initial experiments were performed using Google Colab GPU instances. However, an important limitation was identified during training: datasets stored directly in Google Drive produced significant latency during dataloading operations. Consequently, the dataset had to be copied to local temporary storage before training in order to avoid bottlenecks during image loading and preprocessing.

As the experiments evolved, larger architectures and longer training schedules significantly increased training times. In particular, experiments involving larger image resolutions and medium-sized YOLO architectures required substantially more computational time.

To address these limitations, later experiments were migrated to Kaggle GPU environments. This transition enabled longer uninterrupted training sessions and overnight experimentation, which became increasingly important when training medium-sized models such as YOLOv8m and YOLO26m.

V. RESULTS

A. Dataset Configuration Comparison

The first experiments aimed to determine which dataset representation was the most suitable for the waste detection task. Three alternative configurations were evaluated using the YOLOv8s architecture:

- Full-category dataset.
- Supercategory dataset.
- Binary dataset.

Table I summarizes the results obtained for each configuration. The binary dataset clearly outperformed both the category and supercategory configurations. While the full-category and supercategory datasets achieved mAP50 values of approximately 0.18, the binary configuration achieved a substantially higher mAP50 of 0.39 and a recall of 0.41.

This result confirms the observations made during the dataset analysis. The strong class imbalance and the limited number of samples available for many classes make multi-class detection considerably more difficult. By reducing the problem to a binary detection task, the model can focus on learning robust object localization features rather than discriminating between a large number of poorly represented classes. Consequently, all subsequent experiments were conducted using the binary dataset configuration.

B. YOLOv8 Architecture Scaling

After selecting the binary dataset, larger YOLOv8 [4] architectures were evaluated to determine whether increasing model capacity could improve detection performance.

The initial YOLOv8s baseline achieved an mAP50 of 0.326 and a recall of 0.288 on the test set. Replacing the model with YOLOv8m increased mAP50 to 0.379 and recall to 0.352.

These results indicate that increasing model capacity provides measurable improvements. However, the gains remained relatively limited compared to the increase in computational cost, suggesting that the primary bottleneck may be related to dataset characteristics rather than insufficient model complexity.

C. YOLO26 Evaluation

Although YOLOv8 [4] provided stable and reproducible results, the overall detection performance remained below the desired level. This motivated the exploration of the more recent YOLO26 architecture.

The motivation behind this decision was the challenging nature of the TACO dataset. Many waste instances are small, partially occluded and embedded within cluttered backgrounds. Since YOLO26 [5] introduces architectural modifications designed to improve feature extraction efficiency, it was hypothesized that the model could provide stronger localization performance under these conditions.

At the native training resolution of 640 pixels, YOLO26m already outperformed YOLOv8m. The best YOLO26m configuration achieved an mAP50 of 0.494, compared to 0.435 for YOLOv8m under similar conditions.

These results suggest that the architectural improvements introduced in YOLO26 [5] are beneficial for waste detection tasks involving small and visually challenging objects.

D. Resolution Analysis

Given the large number of small objects present in the dataset, image resolution was identified as a potentially important factor affecting detection performance. Experiments were

therefore conducted using image sizes of 640, 960 and 1280 pixels.

Increasing the input resolution consistently improved detection performance for both YOLOv8m and YOLO26m. The strongest improvement was observed when increasing the resolution from 640 to 960 pixels. For example, YOLO26m with backbone freezing improved from an mAP50 of 0.494 at 640 pixels to 0.561 at 960 pixels. This behavior is consistent with the dataset analysis.

Since many waste objects occupy only a small region of the image, higher input resolutions preserve more visual information and allow the detector to better localize small instances. However, increasing the resolution beyond 960 pixels was not as effective as expected. Although 1280-pixel models remained competitive, the improvement in detection accuracy was relatively small compared to the substantial increase in computational cost.

E. Freeze versus Full Fine-Tuning

The final experimental factor investigated was the impact of backbone freezing. Despite the fact that this factor was initially selected in order to ease the training of the model, for most configuration, freezing the pretrained backbone produced slightly higher recall and mAP50 values than full fine-tuning. The effect was particularly visible for YOLO26m, where the frozen configuration consistently outperformed the non-frozen counterpart across all tested resolutions.

These results suggest that preserving the generic visual representations learned from COCO [3] provides a useful regularization effect when training on a relatively small dataset such as TACO [1].

F. Best Model

All the results are summarized in Table II, where it can be observed that the best overall configuration was obtained using YOLO26m with backbone freezing and an image resolution of 960 pixels. This model achieved:

- Precision: 0.652
- Recall: 0.517
- mAP50: 0.561
- mAP50-95: 0.411

Compared to the initial binary baseline, this configuration improved mAP50 by approximately 42%, demonstrating the effectiveness of the iterative optimization process.

VI. DISCUSSION

In addition to detection accuracy, inference speed is an important factor for real-time deployment. The reported inference time corresponds to the sum of preprocessing, model inference, and postprocessing times measured per image. The relationship between inference time and frame rate can be expressed as:

$$FPS = \frac{1000}{t_{\text{inf}} \text{ (ms)}}$$

where t_{inf} is the total processing time in milliseconds per image. The selected YOLO26m model (960 px, frozen backbone) achieved an inference time of approximately 45.9 ms, corresponding to 21.8 FPS. Although higher frame rates are desirable, this performance remains within the range typically considered suitable for real-time computer vision applications.

Increasing the input resolution to 1280 pixels produced only moderate improvements in detection performance while substantially increasing inference time to approximately 70–86 ms per image (12–14 FPS). Consequently, the 960-pixel configuration provides the best trade-off between detection accuracy and real-time performance.

VII. FUTURE WORK

Several directions could be explored to further improve detection performance. First, the dataset could be expanded by merging TACO with other waste detection datasets or by manually annotating additional images, increasing both the number of training samples and class diversity. A larger dataset would help reduce overfitting and improve model generalization. Second, a more systematic hyperparameter optimization could be performed, including the exploration of larger batch sizes, alternative optimizers, learning rate schedules. Finally, we could also investigate more advanced architectures, such as transformer-based detectors, which have shown strong performance in challenging object detection tasks and may better capture complex contextual information present in cluttered waste scenes.

VIII. CONCLUSION

This project evaluated YOLO-based object detectors for real-time waste detection using the TACO dataset. Experimental results showed that the binary dataset configuration significantly outperformed the category and supercategory alternatives.

Among all evaluated models, YOLO26m with a frozen backbone and a resolution of 960 pixels achieved the best performance, obtaining an mAP50 of 0.561 while maintaining real-time inference capabilities.

The results indicate that dataset characteristics, particularly class imbalance and the presence of small objects, represent the main limitations for further performance improvements.

REFERENCES

- [1] P. Proença and P. Simões, “TACO: Trash Annotations in Context for Litter Detection”, 2020, Available: <http://tacodataset.org/>
- [2] Ultralytics, “Ultralytics YOLO Documentation,” Available: <https://docs.ultralytics.com>
- [3] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” European Conference on Computer Vision (ECCV), 2014.
- [4] “YOLO by Ultralytics,” GitHub Repository, 2023. Available: <https://github.com/ultralytics/ultralytics>
- [5] Ultralytics, “YOLO26 Documentation,” Available: <https://docs.ultralytics.com/models/yolo26/>

APPENDIX A

ADDITIONAL RESULTS

TABLE I
COMPARISON OF DATASET CONFIGURATIONS USING YOLOv8s.

Dataset	Precision	Recall	mAP50	mAP50-95
Categories	0.407	0.185	0.181	0.110
Binary	0.492	0.412	0.394	0.218
Supercategories	0.465	0.176	0.178	0.107

TABLE II
COMPARISON OF YOLOv8 AND YOLO26 MODELS UNDER DIFFERENT RESOLUTIONS AND FREEZING STRATEGIES.

Model	Resolution	Freeze	Precision	Recall	mAP50	mAP50-95	Total (ms)
YOLO26m	640	Yes	0.695	0.434	0.494	0.350	21.17
YOLO26m	640	No	0.681	0.407	0.471	0.335	21.24
YOLOv8m	640	Yes	0.579	0.428	0.435	0.299	18.72
YOLOv8m	640	No	0.584	0.408	0.437	0.308	19.77
YOLO26m	960	Yes	0.652	0.517	0.561	0.411	46.54
YOLO26m	960	No	0.693	0.436	0.517	0.377	46.91
YOLOv8m	960	Yes	0.662	0.470	0.504	0.368	40.33
YOLOv8m	960	No	0.668	0.412	0.455	0.320	41.23
YOLO26m	1280	Yes	0.606	0.484	0.510	0.367	87.38
YOLO26s	1280	Yes	0.778	0.450	0.529	0.384	39.46
YOLOv8m	1280	Yes	0.616	0.469	0.500	0.362	75.44
YOLOv8s	1280	Yes	0.595	0.442	0.476	0.329	38.36

TABLE III
HYPERPARAMETER SETTINGS ACROSS ALL TRAINING RUNS ("—" STANDS FOR DEFAULT VALUES).

Model	Dataset	Epochs	imgsz	Batch	Optimizer	lr0	WD	Freeze
YOLOv8s	binary	25	640	32	—	—	—	—
YOLOv8s	categories	25	640	32	—	—	—	—
YOLOv8s	supercategories	25	640	32	—	—	—	—
YOLOv8s	binary	100	640	16	AdamW	0.001	0.0005	—
YOLOv8m	binary	100	640	16	AdamW	0.001	0.0005	—
YOLOv8m	binary	100	640	32	AdamW	0.001	0.0005	—
YOLOv8m	binary	100	960	32	AdamW	0.001	0.0005	—
YOLOv8m	binary	100	960	32	AdamW	0.001	0.0005	10
YOLO26m	binary	100	640	32	AdamW	0.001	0.0005	—
YOLO26m	binary	100	640	32	AdamW	0.001	0.0005	11
YOLO26m	binary	100	960	32	AdamW	0.001	0.0005	—
YOLO26m	binary	100	960	32	AdamW	0.001	0.0005	11
YOLOv8m	binary	100	1280	16	AdamW	0.001	0.0005	10
YOLOv8s	binary	100	1280	16	AdamW	0.001	0.0005	10
YOLO26m	binary	100	1280	16	AdamW	0.001	0.0005	11
YOLO26s	binary	100	1280	16	AdamW	0.001	0.0005	11